

Programación Distribuida y Tiempo Real

Trabajo Práctico N°2

Austin Myles Barker

Nicolas Bonoris

29 de septiembre de 2025

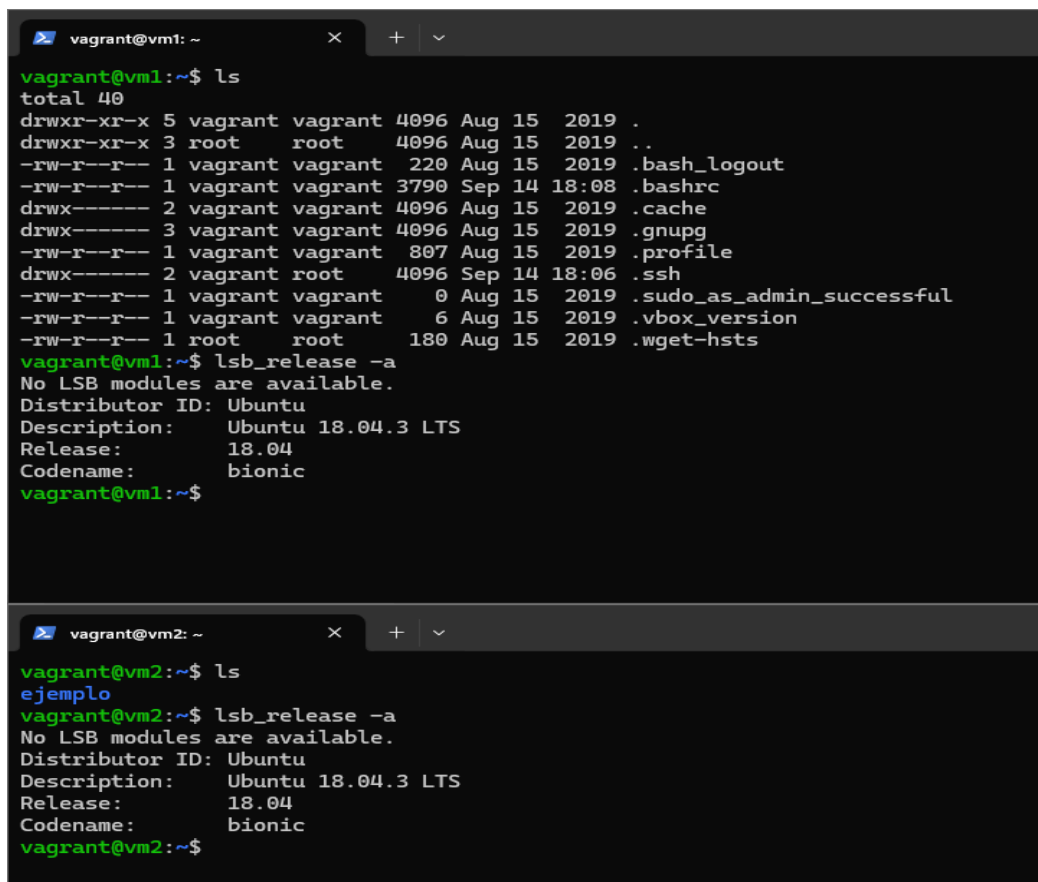
Documentación scripts adjuntos:

Todos son archivos .c adjuntos con la entrega, se explican en el presente documento en su gran mayoría cuando es oportuno, debajo una mínima descripción individual de cada archivo a forma de guía.

- **ping:** Mide el tiempo de Cliente haciendo el Ping (envío y recepción).
- **pong:** Mide el tiempo de Servidor haciendo el Pong (recepcion y envío).
- **auto_cl:** Automatización del proceso de envío del cliente.
- **auto_sv:** Automatización del proceso de recepción del servidor.
- **cliente:** Script utilizado en el automatizador. Se conecta con el server persistente.
- **persistent:** Script utilizado en el automatizador, inicia un servidor persistente.
- **times:** Script de medición de tiempo de servidor básico.
- **timec:** Script de medición de tiempo de cliente básico.

1. **Buscar y utilizar en vagrant una versión más reciente de Ubuntu LTS que la dada en la explicación de práctica. Identificar versión, origen y si tuviera diferencias, explicar brevemente. Mostrar con una captura de pantalla la identificación en VirtualBox de la máquina virtual y el inicio de la terminal en funcionamiento. Consulte a un asistente de IA (deje constancia de cuál) si hay alguna versión en particular que puede ser mejor que otra y por qué.**

Capturas:



The image shows two terminal windows from a Vagrant environment. The top window, titled 'vagrant@vm1: ~', shows the output of 'ls' and 'lsb_release -a'. The bottom window, titled 'vagrant@vm2: ~', shows the output of 'ls' and 'lsb_release -a'. Both windows confirm the system is Ubuntu 18.04.3 LTS with the codename 'bionic'.

```
vagrant@vm1: ~$ ls
total 40
drwxr-xr-x 5 vagrant vagrant 4096 Aug 15 2019 .
drwxr-xr-x 3 root    root    4096 Aug 15 2019 ..
-rw-r--r-- 1 vagrant vagrant 220 Aug 15 2019 .bash_logout
-rw-r--r-- 1 vagrant vagrant 3790 Sep 14 18:08 .bashrc
drwx----- 2 vagrant vagrant 4096 Aug 15 2019 .cache
drwx----- 3 vagrant vagrant 4096 Aug 15 2019 .gnupg
-rw-r--r-- 1 vagrant vagrant 807 Aug 15 2019 .profile
drwx----- 2 vagrant root    4096 Sep 14 18:06 .ssh
-rw-r--r-- 1 vagrant vagrant 0 Aug 15 2019 .sudo_as_admin_successful
-rw-r--r-- 1 vagrant vagrant 6 Aug 15 2019 .vbox_version
-rw-r--r-- 1 root    root    180 Aug 15 2019 .wget-hsts
vagrant@vm1:~$ lsb_release -a
No LSB modules are available.
Distributor ID: Ubuntu
Description:    Ubuntu 18.04.3 LTS
Release:        18.04
Codename:       bionic
vagrant@vm1:~$
```

```
vagrant@vm2: ~$ ls
ejemplo
vagrant@vm2:~$ lsb_release -a
No LSB modules are available.
Distributor ID: Ubuntu
Description:    Ubuntu 18.04.3 LTS
Release:        18.04
Codename:       bionic
vagrant@vm2:~$
```

¿Hay alguna versión mejor en particular?:

ChatGPT: la box con una versión más reciente de Ubuntu de momento es la ubuntu/jammy64, ó JammyJellyfish, que se asemeja a la versión 22.04 de Ubuntu LTS. Este, al ser una versión más reciente del Kernel de linux, a diferencia de la versión 18.04, posee librerías que no están incluidas en bionic64. Este se expone a las posibles vulnerabilidades parcheadas en versiones anteriores de Ubuntu. Vale aclarar que de la versión actual de Ubuntu LTS, la versión 24.04, no existen boxes oficiales hechos por el equipo de desarrolladores de Ubuntu, y no hay noticias recientes a la fecha sobre la producción de boxes de esa versión del Kernel.

Gemini: No hay una versión LTS de Ubuntu que sea universalmente "mejor" que las otras, ya que la elección a menudo depende de tus necesidades específicas de hardware y software, y de tu preferencia por el equilibrio entre estabilidad y actualidad. Aunque todas las versiones LTS son estables, las más recientes suelen ofrecer mejor compatibilidad con hardware moderno y versiones más actualizadas de software clave.

- **Versiones recientes (ej. 24.04 LTS y 22.04 LTS):** Suelen ser las **mejores opciones actuales** porque son estables y, a la vez, tienen mejor soporte para el hardware más nuevo (gracias a *kernels* y controladores más recientes).
- **Versiones más antiguas (ej. 20.04 LTS):** Pueden ser mejores para **hardware más antiguo** o si dependes de software específico que no ha sido actualizado para las versiones más recientes de las librerías o dependencias.

En nuestro análisis, claramente no hay una versión "mejor" en particular, sino que va a depender en lo que uno valore como parámetros importantes. Si uno busca adaptarse a hardware y funcionalidades más modernos, posiblemente deba buscar una versión más nueva, que suele llevar consigo un soporte más extendido y bugs/problemas menos conocidos y documentados. En cambio, para casos y componentes de hardware más específicos, podemos llegar a necesitar una versión más antigua, la cual también posiblemente tenga más documentación y casos de error más conocidos.

2. **Ejecutar los experimentos del ejercicio 4 a), b) y c) de la práctica anterior en dos computadoras físicamente diferentes. Se copia aquí el enunciado para mayor comodidad: comunicar 10^1 , 10^2 , 10^3 , 10^4 , 10^5 y 10^6 bytes desarrollando experimentos que muestre el tiempo que toma en el cliente proveer una descripción mínima de las computadoras usadas (CPU, RAM, OS). Aclare si las computadoras están en la misma red local o en dos redes locales diferentes (en este último caso, es posible que deban modificar al menos un router si ambas computadoras tienen acceso vía NAT). Pueden utilizar las computadoras del aula o sus propias computadoras. Documentar y entregar no solamente el código de los procesos de comunicaciones sino también el propio experimento.**

- a. **La función `write(...)` para cada cantidad de datos. Tomando C como**

ejemplo:

t0 = ...

write(...)

t1 = ...

tiempo = t1 - t0;

b. La función read(...). Tomando C como ejemplo:

t0 = ...

n = read(sockfd, buffer, 255);

t1 = ...

tiempo = t1 - t0;

c. Grafique y explique los resultados obtenidos. En particular, identifique si las diferencias de tiempos son proporcionales a las cantidades de datos, por ej: ¿write(...) con 1000 bytes toma 10 veces más tiempo que con 100 bytes? Identifique si el tiempo de la función read() se mantiene constante, dado que involucra siempre la misma cantidad de datos.

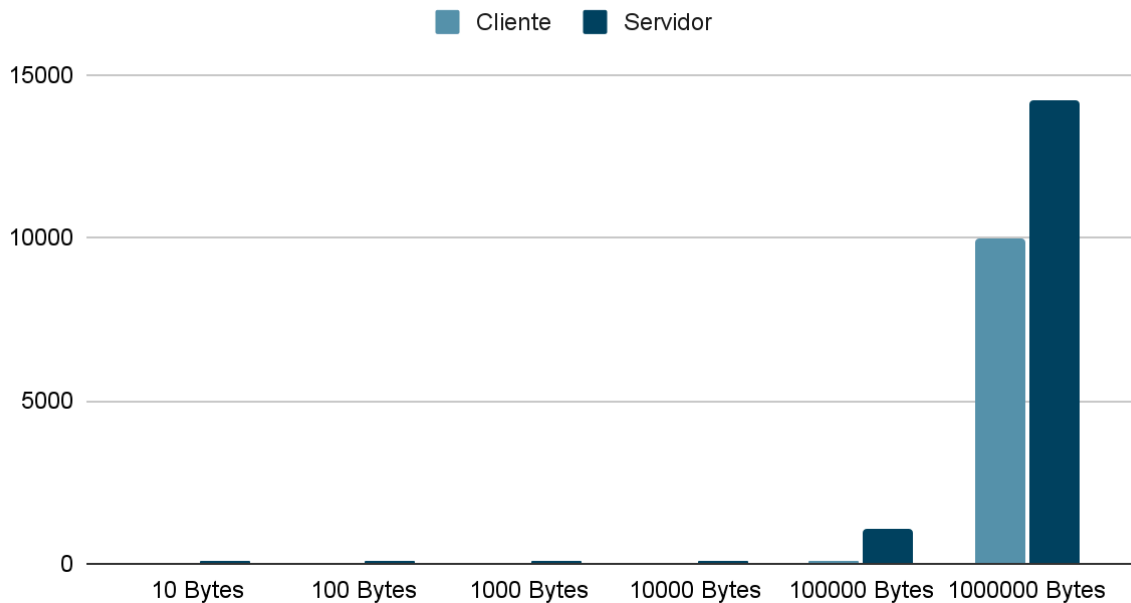
Las computadoras se encuentran en una red local. Se utilizaron las computadoras del aula 8, sala de PC, cuyas especificaciones son:

- i. **CPU:** Intel(R) Core(TM) i5-6400 CPU @ 2.70GHz
RAM: 8GB
SO: Ubuntu 20.04.6 LTS
- ii. **CPU:** Intel(R) Core(TM) i5-6400 CPU @ 2.70GHz
RAM: 8GB
SO: Ubuntu 20.04.4 LTS

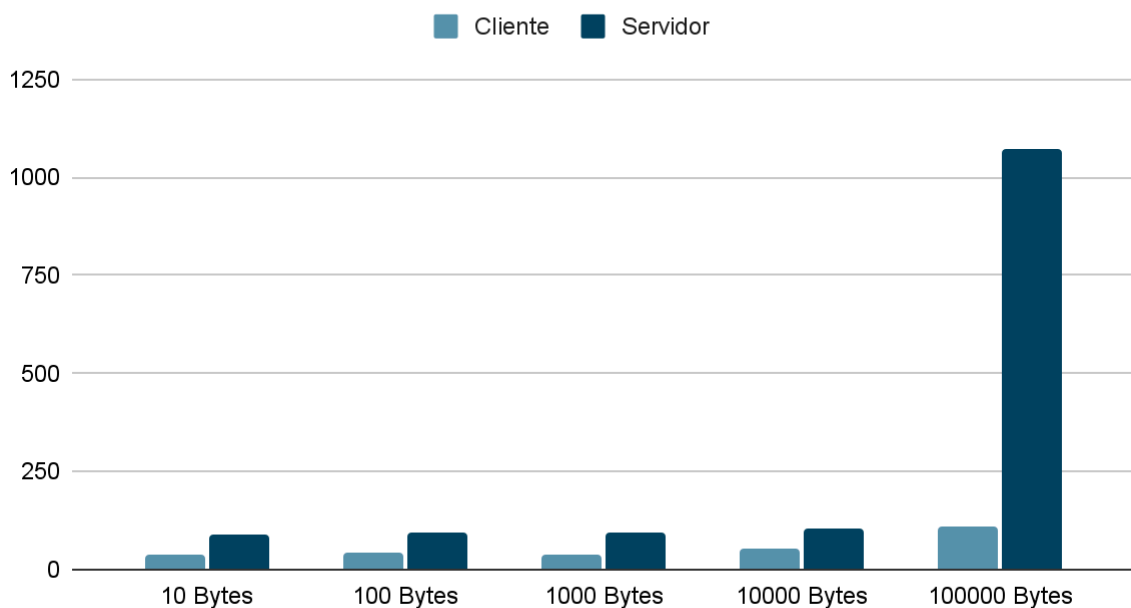
Mediciones en microsegundos (μ s)

Tamaño de Buffer	Tiempos para Cliente	Tiempos para Servidor
10¹	39.333 ms	87.900 ms
10²	40.857 ms	93.500 ms
10³	39.000 ms	92.100 ms
10⁴	54.000 ms	105.700 ms
10⁵	108.000 ms	1071.000 ms
10⁶	9972.000 ms	14215.000 ms

Tiempos de Cliente/Servidor según tamaño de buffer en μs



Tiempos de Cliente/Servidor según tamaño de buffer en μs



Mismo gráfico pero limitado hasta resultados de 10^5 para poder apreciar mejor los resultados menores.

- El crecimiento del tiempo del cliente y del servidor en un principio no mantiene un crecimiento proporcional en relación a la cantidad de datos enviados al buffer. Para los tamaños 10^1 , 10^2 , 10^3 y 10^4 se mantiene casi similar con un crecimiento casi lineal, ya una vez pasada la cifra de 10^5 , notamos que el tiempo en el cliente se

duplica mientras que en el tiempo del servidor el tiempo crece 10 veces. Por último, al estar con 10^6 vemos que la relación tuvo un crecimiento exponencial.

- La operación read en sí tiene un comportamiento diferente según la cantidad enviada, con buffer de tamaño reducido el tiempo de lectura se mantiene medianamente consistente dado que lee todo el buffer en una sola lectura. Al ir aumentando el tamaño del buffer a por ejemplo 10^5 , ya encontraremos diferencias en los tiempos de lectura y cantidad de iteraciones necesarias para poder leer el buffer completo. Esto es debido a los diferentes factores como la disponibilidad de datos en el buffer TCP, la fragmentación de los paquetes de gran tamaño que superan el MTU de la red, el tamaño de ventana de recepción TCP y la congestión de la red.

```
alumnos@Aula8-10:~/Downloads/Otin$ ./time 2020 10000
Cantidad leída en esta iteración: 10000 bytes
Tiempo transcurrido del servidor: 234.000 microsegundos
alumnos@Aula8-10:~/Downloads/Otin$ |

alumnos@Aula8-10:~/Downloads/Otin$ ./time 2020 100000
Cantidad leída en esta iteración: 13032 bytes
Cantidad leída en esta iteración: 1448 bytes
Cantidad leída en esta iteración: 4344 bytes
Cantidad leída en esta iteración: 5792 bytes
Cantidad leída en esta iteración: 14480 bytes
Cantidad leída en esta iteración: 17376 bytes
Cantidad leída en esta iteración: 28960 bytes
Cantidad leída en esta iteración: 14568 bytes
Tiempo transcurrido del servidor: 1405.000 microsegundos
alumnos@Aula8-10:~/Downloads/Otin$ |
```

3. Desarrollar los mismos experimentos de comunicaciones que en el caso anterior, con las mismas computadoras, pero ahora con la misma cantidad de datos en un sentido y en el inverso (experimento “ping-pong”). El tiempo total “de ida y vuelta” de los datos dividido por 2 es el que se utiliza para una estimación del tiempo de mensajes en una dirección. Comparar los tiempos del ejercicio anterior con los que se obtienen en este experimento para cada cantidad de bytes.

Aclaración consultada previamente con el profesor Tinetti, los tiempos recibidos desde el ping y el pong poseen una diferencia a la cual no fue posible encontrar una causa, así que se tomaron los tiempos de ambos scripts por separado.

PING –TCP

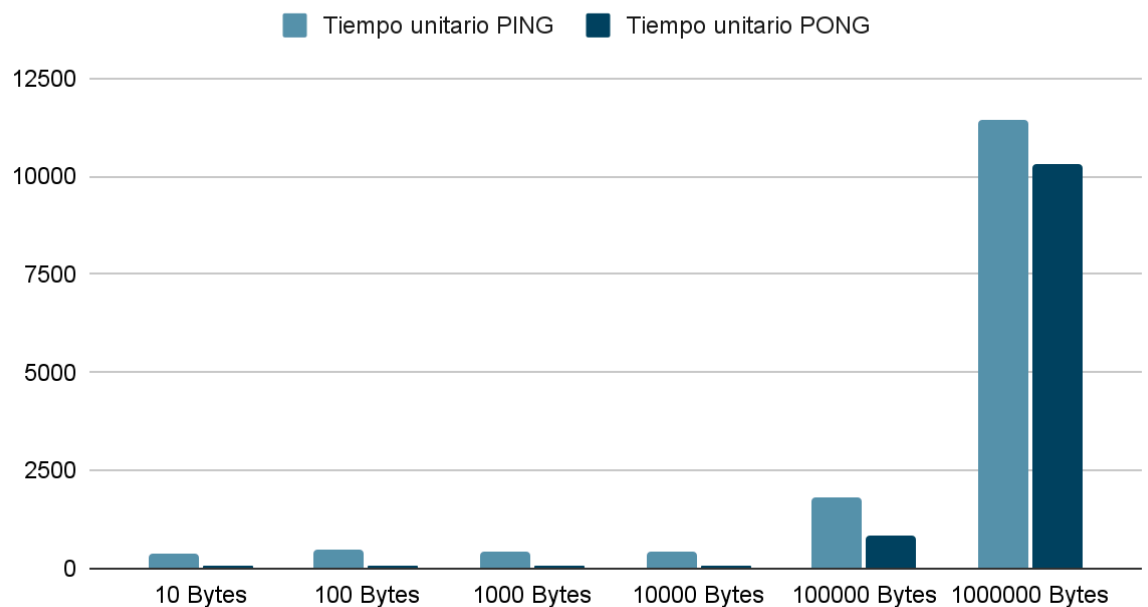
Tamaño Buffer	Tiempo Total	Tiempo unitario
10^1	791.000 ms	395.500 ms
10^2	920.000 ms	460.500 ms
10^3	908.000 ms	454.000 ms

Tamaño Buffer	Tiempo Total	Tiempo unitario
10^4	849.000 ms	424.500 ms
10^5	3663.000 ms	1831.500 ms
10^6	22925.000 ms	11462.500 ms

PONG – ICMP

Tamaño Buffer	Tiempo Total	Tiempo unitario
10^1	153.000 ms	76.500 ms
10^2	146.000 ms	73.000 ms
10^3	138.000 ms	69.000 ms
10^4	173.000 ms	86.500 ms
10^5	1716.000 ms	858.000 ms
10^6	20578.000 ms	10289.000 ms

Tiempos unitarios en microsegundos por tamaño de buffer

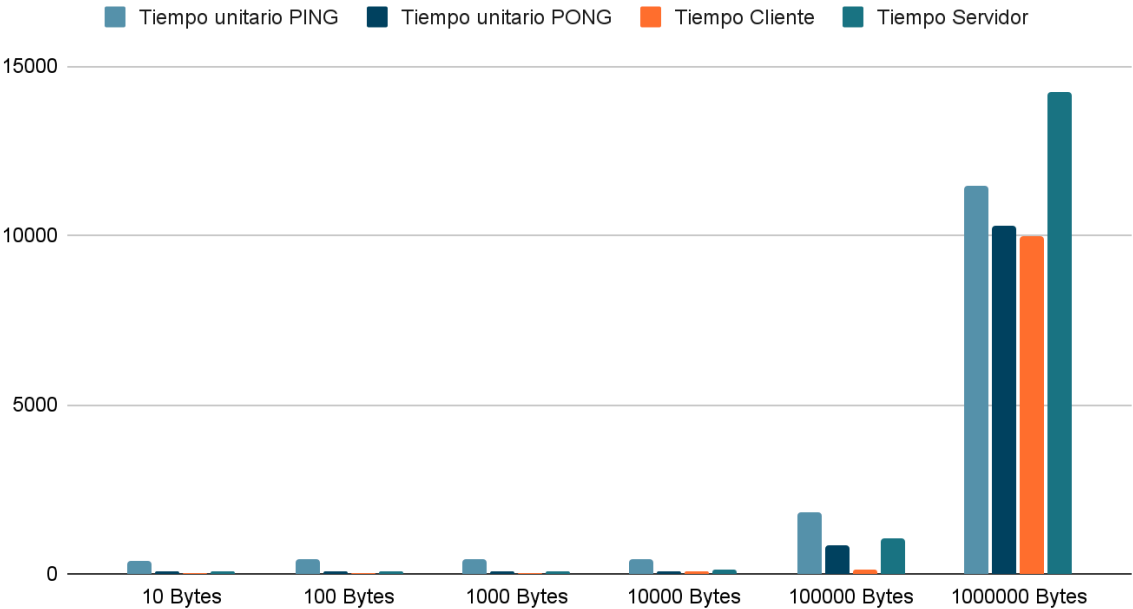


Comparación de tiempo con el script del experimento anterior

Tamaño Buffer	Tiempo unitario PING	Tiempo Cliente
10 ¹	395.500 ms	39.333 ms
10 ²	460.500 ms	40.857 ms
10 ³	454.000 ms	39.000 ms
10 ⁴	424.500 ms	54.000 ms
10 ⁵	1831.500 ms	108.000 ms
10 ⁶	11462.500 ms	9972.000 ms

Tamaño Buffer	Tiempo unitario PONG	Tiempo Servidor
10 ¹	76.500 ms	87.900 ms
10 ²	73.000 ms	93.500 ms
10 ³	69.000 ms	92.100 ms
10 ⁴	86.500 ms	105.700 ms
10 ⁵	858.000 ms	1071.000 ms
10 ⁶	10289.000 ms	14215.000 ms

Tiemops unitarios



4. Teniendo en cuenta los experimentos de tiempos realizados, desarrollar scripts para desplegar un ambiente de experimentación de comunicaciones en una computadora con Vagrant para los siguientes escenarios:
 - a. Dos máquinas virtuales, cada una con un proceso de comunicaciones.
 - b. Una máquina virtual con uno de los procesos de comunicaciones y el otro proceso de comunicaciones en el host.

En todos los casos deberían quedar los resultados disponibles para su posterior análisis. Se debe resolver el problema de “asincronismo” que genera un error si el proceso “cliente” inicia la ejecución antes que el proceso “servidor”. Consultar con un asistente con IA cual es seria el mejor método experimental para estimar el tiempo de comunicaciones entre dos computadoras diferentes y comente si le parece correcto o puede identificar algún problema en la respuesta del asistente con IA.

Para el problema de asincronismo, en el script que correrá el cliente de la conexión, habrá un loop encargado de verificar mediante netcat “nc” si la conexión con el host está disponible o no. Mientras tanto estará loopeando con un sleep interno hasta que el host esté disponible para conectar. La respuesta de la IA sin embargo no propuso soluciones funcionales para el problema de la primera conexión durante la primera iteración. Se hicieron cambios en los scripts de cliente y servidor para realizar las pruebas con una mayor velocidad, en este caso el cliente recibe la cantidad de iteraciones por parámetro y el servidor a su vez fue estructurado para ser persistente y cerrar después de cierta cantidad de iteraciones. Los scripts exportan su output en un .csv que está estructurado por número de iteración como fila y tamaño de buffer como columna. Los resultados están expresados en microsegundos.

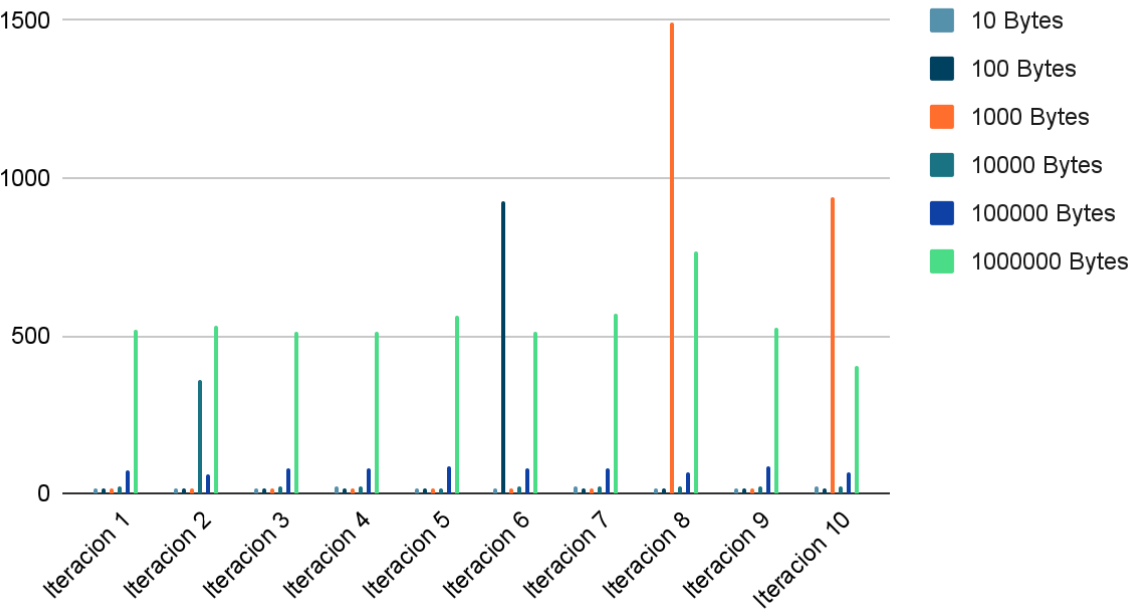
Para el **escenario 1** se utilizó el Vagrant proporcionado por la cátedra.

Servidor:

Iteración	10 ¹	10 ²	10 ³	10 ⁴	10 ⁵	10 ⁶
1	19	16	16	24	75	522
2	20	16	20	361	63	531
3	20	17	20	23	83	516
4	21	17	15	24	82	515
5	20	17	14	18	84	564
6	20	926	14	24	82	511
7	21	16	18	25	78	573
8	20	17	1496	23	70	766

9	18	16	16	22	87	526
10	21	17	940	22	65	406

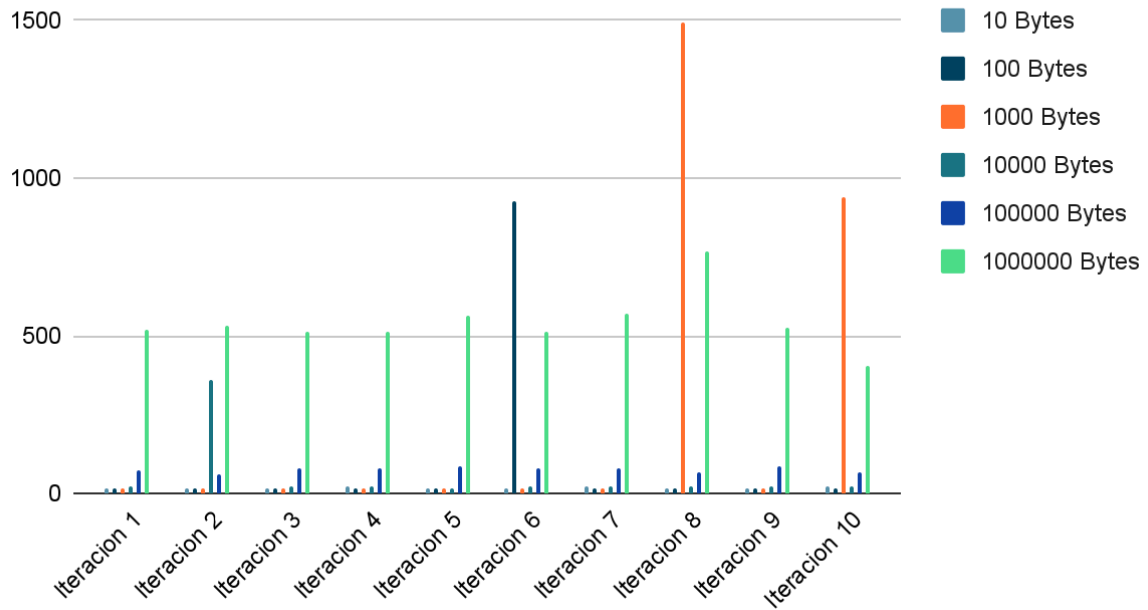
Tiempos por iteraciones y tamaño de buffer para servidor



Cliente:

Iteración	10 ¹	10 ²	10 ³	10 ⁴	10 ⁵	10 ⁶
1	325	470	404	593	475	1026
2	434	489	384	325	360	1039
3	419	477	469	433	559	1018
4	434	579	323	662	551	1028
5	416	496	356	369	523	1070
6	419	308	316	625	546	1018
7	476	516	525	539	481	1248
8	470	517	659	513	589	896
9	376	453	417	554	516	1033
10	427	524	497	512	370	746

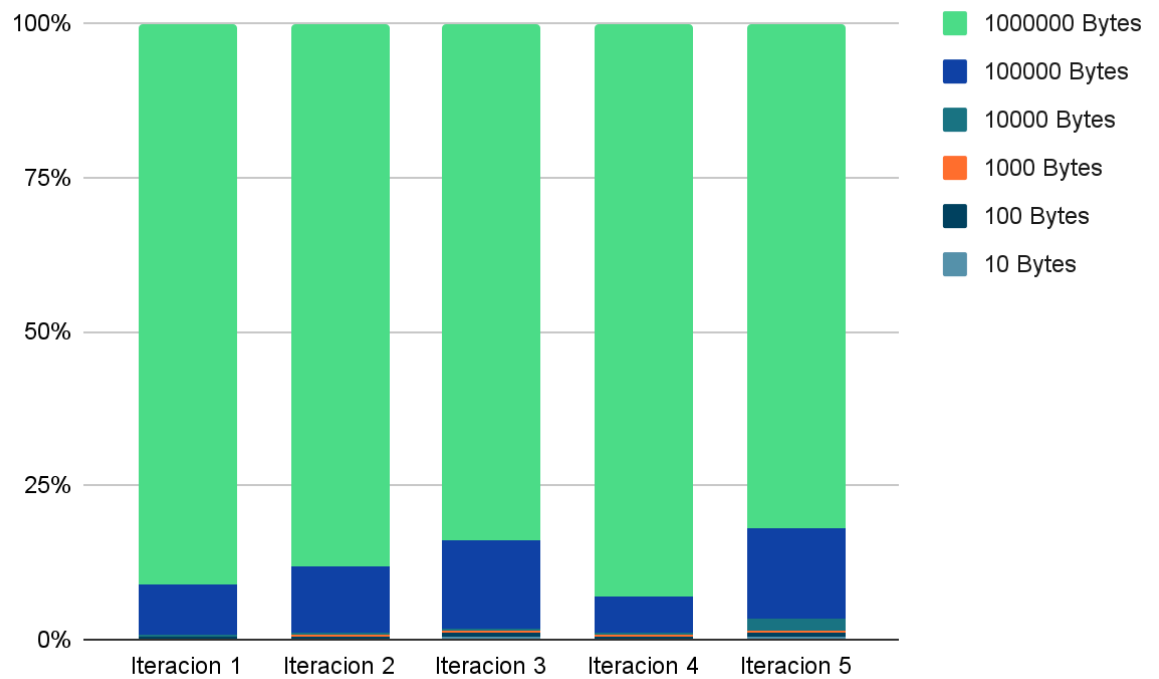
Puntuación lograda



Para el **escenario 2** se utilizó una de las vm creadas por el vagrantfile de la cátedra y el WSL Ubuntu en mi maquina propia

Ubuntu - Cliente:

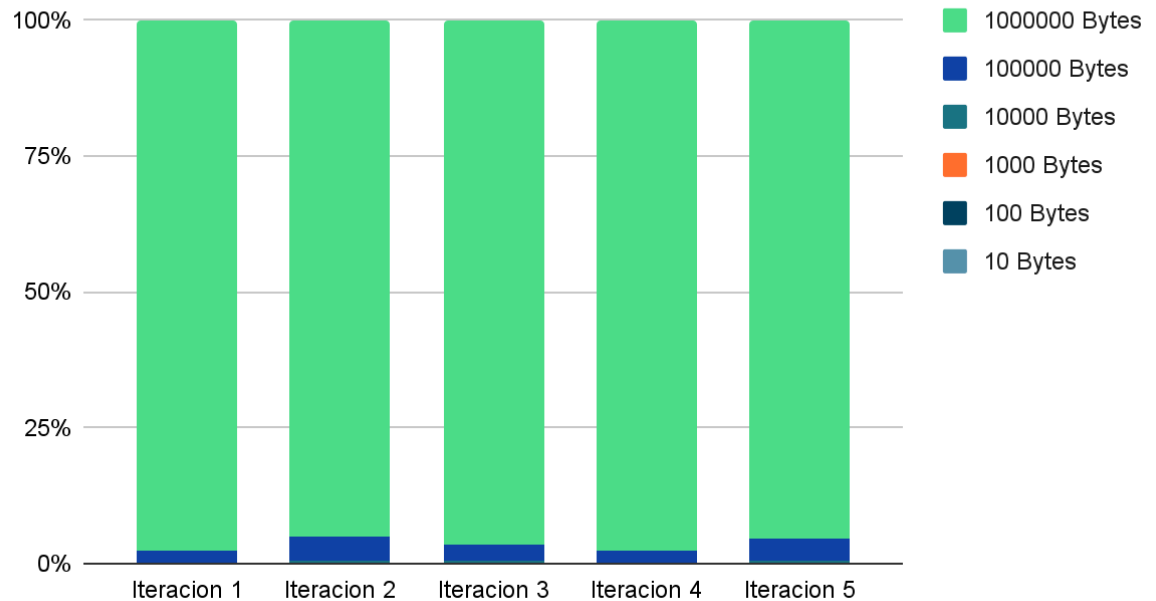
Iteración	10 ¹	10 ²	10 ³	10 ⁴	10 ⁵	10 ⁶
1	19	19	22	24	822	9062
2	18	17	17	27	799	6535
3	21	15	12	17	494	2916
4	15	29	19	26	431	6911
5	15	23	17	79	552	3139



Vagrant VM1 - Servidor:

Iteración	10 ¹	10 ²	10 ³	10 ⁴	10 ⁵	10 ⁶
1	11	11	11	19	343	17474
2	11	10	12	19	521	11006
3	12	11	11	20	354	10790
4	7	14	10	19	336	15282
5	7	11	8	14	426	9802

Tiempos de segundo experimento Vagrant-Host



En cuanto al mejor método de evaluación de tiempos de comunicaciones, según ChatGPT: “El mejor método experimental es **medir el round-trip time (RTT)**: enviar un mensaje pequeño desde una máquina a la otra, recibir la respuesta y medir la diferencia de tiempo; repetir varias veces y promediar. Herramientas comunes: **ping** (para latencia) o **iperf** (para latencia y throughput)”.

Es una respuesta que tiene sentido, ya que sería fundamental que el protocolo en sí no añada mucha latencia, o no pueda generar casos de segmentación de paquetes (ya que en ese caso tendríamos datos no muy fieles y útiles para lo que estamos buscando medir), además, nos hace mucho sentido el hecho de hacer una cantidad considerable de pruebas y promediarlas, ya que esto nos dará datos mucho más fieles a lo que deberíamos esperar en una situación aleatoria. Como último punto, ping e iperf son buenas opciones también, ya que son herramientas muy utilizadas, casi un estándar en las comunicaciones de red.

5. Explique si los resultados de tiempo de comunicaciones del experimento del ej. anterior se ven afectados por el orden de ejecución y las diferencias de tiempo de ejecución iniciales de los programas utilizados ¿qué sucede si por alguna razón hay una diferencia de 10 segundos en el inicio de la ejecución entre un programa y otro? Sugerencia: incluya un “sleep” de 10 segundos entre la ejecución de diferentes partes de los programas y comente.

En el programa inicialmente se optó por la estrategia de que el script de automatización del cliente tenga varios “sleep” en puntos vitales de la ejecución para evitar problemas relacionados a la conexión con el servidor.

En sí los tiempos que se toman para crear el output no varían, la única diferencia notable en este caso es que el script de automatización por parte del servidor y el

cliente toman mucho más tiempo. Aunque esto es algo positivo ya que en nuestro caso le da tiempo al socket para que no se generen errores de conexión, cosa que pasa si eliminamos de ambos algoritmos los **“sleep”**.